

Designing a Graphics Processing Unit for 2D Rendering on FPGA for Educational Purpose

Rupak Saha

Dept. of Computer Science & Engineering
Rajshahi University of Engineering & Technology
Rajshahi-6204, Bangladesh
rupak10987@gmail.com

Nahin UL Sadad

Dept. of Computer Science & Engineering
Rajshahi University of Engineering & Technology
Rajshahi-6204, Bangladesh
nahinsd100@gmail.com

Md. Nazrul Islam Mondal

Dept. of Computer Science & Engineering
Rajshahi University of Engineering & Technology
Rajshahi-6204, Bangladesh
nimbd109@gmail.com

Abstract—In this paper, design and implementation of a 2D graphics processing hardware unit using FPGA for educational purpose is presented. We propose a more simple & minimalist GPU architecture, which is flexible and easy to understand than existing industrial GPUs. We present a platform for understanding GPU architecture by implementing of graphics algorithms on FPGA for students to use this in laboratories. Students can be benefited from this work as it can be used as a base understanding tool in terms of GPU architecture. It can also be extended to be used in cross disciplinary applications like 3D rendering, computer vision and signal processing visualisation. This research includes descriptions of hardware realization of 2D rasterization rendering algorithms. Hardware Description Language (HDL) called verilog is utilized in designing the architecture. Xilinx Vivado Design Suite software is used for FPGA implementation, targeting Artix 7 FPGA board, which allows users to comprehend the fundamental ideas of hardware design and their applications in graphics rendering.

Index Terms—2D Graphics, Rasterization, FPGA

I. INTRODUCTION

The development of graphics rendering has fundamentally changed how we engage with digital spaces, influencing everything from simulations and video games to scientific visualization and design. In computer graphics, virtual scenes are either rendered previously and then displayed or created in real time. In terms of creating scene images beforehand and then showing them as frames, is a very time consuming job. It's normally used in movie making. On the other hand computer games require real time rendering, as scenes change pretty frequently. Thus beforehand rendering is impossible in this case.

In real time rendering, mainly two types of rendering approaches exists: Raster and Ray Tracing. In terms of ray tracing, every light ray emerged from a viewpoint in a scene is traced and determined if the ray had hit any object in the scene or not. If it had hit, then the corresponding position of the screen, through which the ray was passed is colored according to the object's color. Then shadow and reflection is applied. Ray tracing takes a huge amount of time. So a more computationally inexpensive algorithm called rasterization or scanline algorithm is usually used [1].

This technique works by projecting 3D objects in 2D view plane (Fig. 1) and then filling the object's structure with filled

triangles or other primitives (polygons) [1]. It is shown in Fig. 2

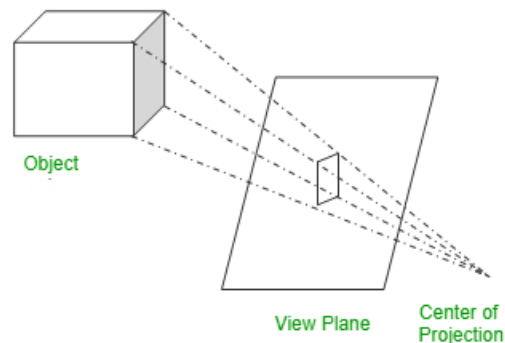


Fig. 1. Perspective Projection [9]

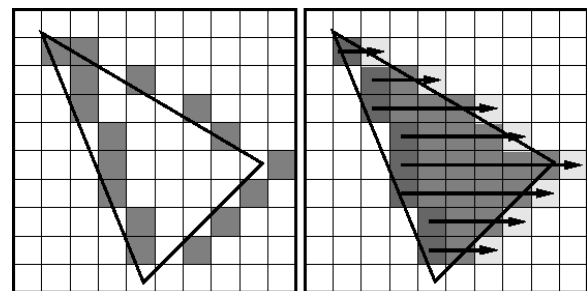


Fig. 2. Raster Procedure by filled triangles with scan line [10]

Modern raster visuals are intricate and complicated, which has sparked a growing interest in finding new rendering and accelerating techniques. Due to its inherent parallel processing powers and reconfigurable facilities, the use of Field-Programmable Gate Arrays (FPGAs) has emerged as a potential route among these techniques [8]. The hardware implementation procedure follows the schemes of Fig. 3. As shown in figure, the process involves translating HDL code into a logical representation, mapping it to target technology, followed by physical placement and routing on the FPGA,

finally generating bitstream and downloading for implementation.

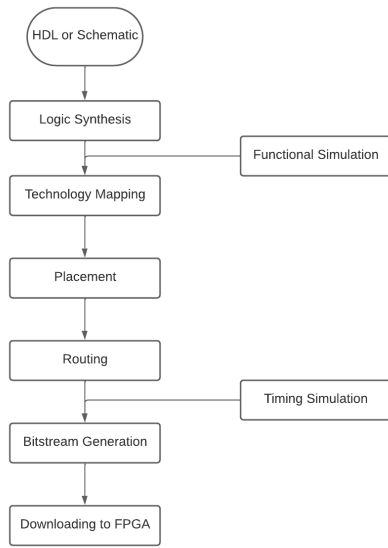


Fig. 3. Hardware Implementation Scheme of Algorithms

In this paper, we present a novel approach to designing a dedicated Graphics Processing Unit (GPU) tailored for 2D rendering tasks, based on the filled triangle rasterization method, implemented on a Field-Programmable Gate Array (FPGA) with a special emphasis on it's simplicity and it's usefulness in educational contexts.

As most industrial level GPU architectures are normally complex and not available for education, our work can be used in laboratory for students to understand GPU architecture easily. The research contributions are as follows:

- We present a platform that implements graphics rendering algorithms using FPGA which renders 2D models. Its simpler and easy to understand for students and amateurs than the existing models which focus only on the rendering but not on it's usefulness in educational purpose.
- Our FPGA-based GPU can find applications in embedded systems where size, power, and performance constraints are essential. So our research can contribute to the development of GPUs suitable for embedded applications.
- Our work can be extended to be used in cross disciplinary applications like 3D rendering, computer vision, signal processing visualisation.

II. RELATED WORKS

Real-time rendering performance and educational objectives may both benefit from the convergence of graphics rendering and Field-Programmable Gate Arrays (FPGAs), which has become an exciting research topic. This survey of the literatures seeks to give an overview of significant contributions to graphics rendering on FPGAs with an emphasis on how they might be used in educational settings.

- Kasik et al. shows how 3 dimensional models were rendered in wireframe representation using FPGA Spartan 6 device [2]. But their work does not render full model.
- Schmittler et al. have constructed a prototype of the whole ray tracing process on a single FPGA chip [3]. Real-time frame rates of 20 to 60 frames per second are achieved while operating at just 90 MHz, and support for texturing, various light sources, and different degrees of reflection or transparency are all included. But it was ray tracing based rendering, which is inherently more complex than raster hardware. Its design is complicated and is not suitable for laboratory use.
- Warner implements a two-dimensional and three-dimensional graphics processor prototype to show that a 3-D graphics processor implementation on an FPGA is viable [4]. A wire frame graphics rendering engine is created using VHDL and the development tools from Xilinx utilizing a Virtex 5 ML506 FPGA development kit. This author also implemented wire frame model rendering, not the colorful whole model and it is more complex than our model.

While these research works were able to implement graphics processing but they lacked simplicity, in terms of educational purpose. Our work provides easy to understand and customizing hardware implementation of underlying 2D graphics processing algorithm which is helpful for students who are eager to learn graphics rendering and FPGA design methods.

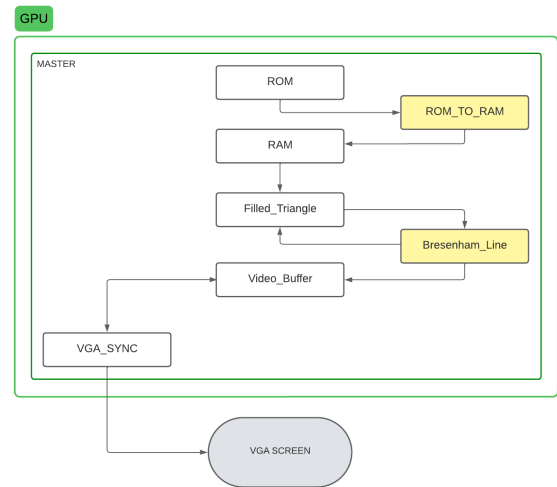


Fig. 4. Proposed Architecture

III. METHODOLOGY

This section discusses the design methodology, from the algorithmic front end to hardware back end. The Proposed architecture depicted in Fig. 4 contains 8 modules ROM, RAM, ROM_TO_RAM, Filled_Triangle, Bresenham_Line, Video_Buffer, VGA_SYNC and MASTER. ROM contains vertex buffer data. ROM_TO_RAM module fetches data from ROM and writes to RAM, Filled_Triangle module fills the

points inside a triangle. It gets data from RAM and it calls Bresenham_Line module to draw horizontal line segments that make up the whole triangle. Also each points in the triangle are written to the Video_Buffer memory which is looked up by VGA_SYNC circuit that draws colored pixels in a VGA screen. The communication between modules is controlled by the MASTER module with the help of some control registers.

Now implementations of individual modules along with their block diagrams and working procedures are discussed below:

A. ROM Module

The ROM module (Fig. 5) stores the vertex buffer data. The vertex data contains information about a 2D model. It contains the vertex positions (V_x , V_y) of each triangle's vertices which can be accessed by the wire read_addr which is a 8 bit long bus. The data stored are 32 bit long. It can only store data, and can only be read from. Any write operation in this module is invalid.

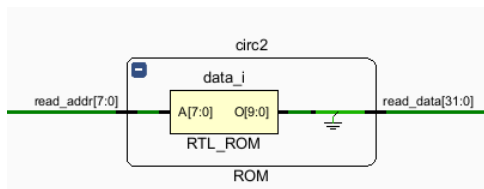


Fig. 5. ROM module circuit diagram

B. RAM Module

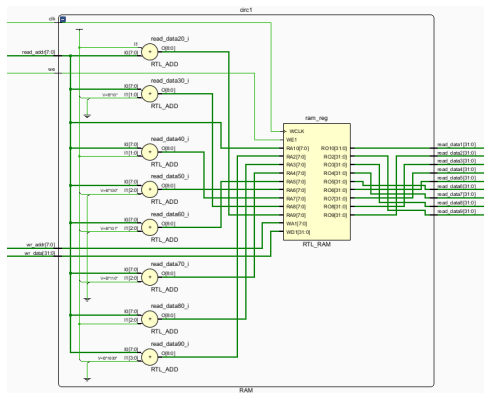


Fig. 6. RAM module circuit diagram

The RAM module (Fig. 6) can do both read and write operations simultaneously. It is fed with data containing one triangle at a time from ROM by ROM_To_RAM module. Once rasterization of one triangle is done, it is fed with the next triangle data from the ROM.

C. ROM_TO_RAM Module

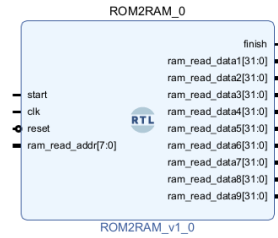


Fig. 7. ROM_TO_RAM module block diagram

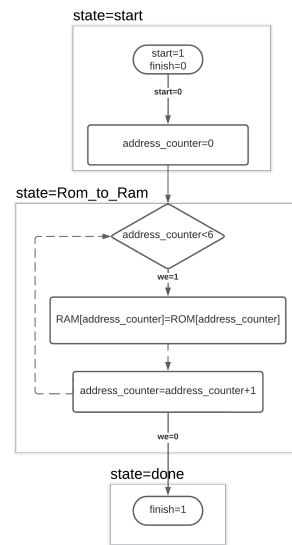


Fig. 8. ROM_TO_RAM module workflow diagram

The ROM_TO_RAM module (Fig. 7) transfers data from ROM to RAM one smallest data unit (vertex components V_x or V_y of the triangles) at a time. It's Work flow is shown in Fig. 8.

D. Bresenham_Line Module

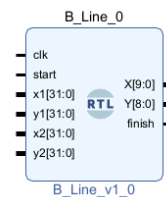


Fig. 9. Bresenham_Line module block diagram

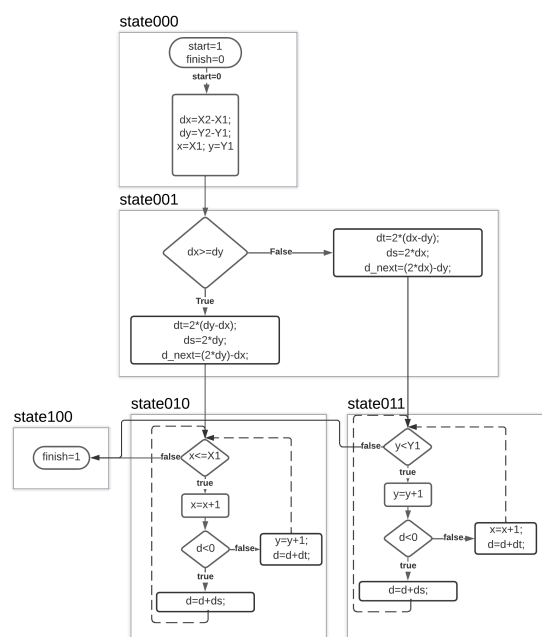


Fig. 10. Bresenham Line module work flow diagram

This module (Fig. 9) takes two end points of a line and outputs interpolated points between the two ends of that line using bresenham line drawing algorithm. It's working procedure is depicted in Fig. 10.

E. Filled Triangle Module

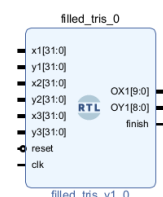


Fig. 11. Filled Triangle module block diagram

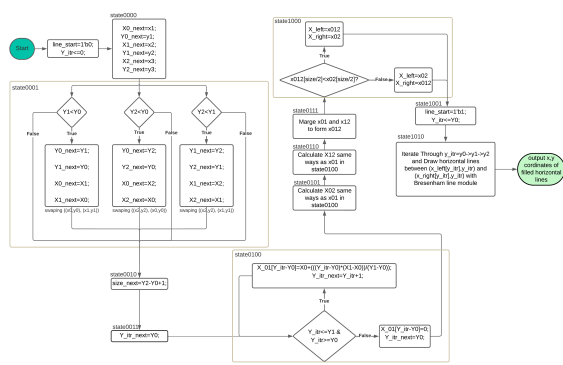


Fig. 12. Filled Triangle module workflow diagram

The Filled_Triangle module (Fig. 11) is fed with the three vertices of each triangle of a model. It outputs the filled pixels coordinates inside the specified triangle, which is written into video buffer memory simultaneously for displaying in the screen later on. Its workflow is shown in Fig. 12.

F. Video Buffer Module

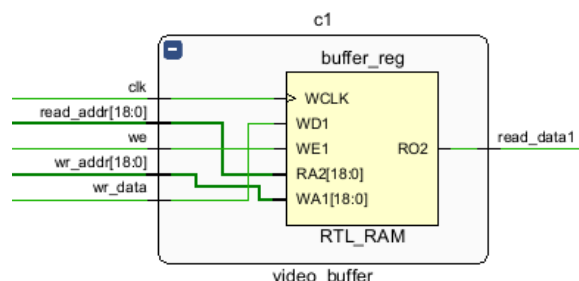


Fig. 13. Video Buffer module circuit diagram

The Video_Buffer module (Fig. 13) is fed with the pixels that need to be drawn on the display. Each pixel position is encoded with a unique address scheme as Pixel address = P_X[9:0], P_Y[8:0] Where P_X and P_Y are the pixel's X and Y coordinates consecutively. Now if a pixel needs to be colored, the corresponding address at the memory for the pixel is set to 1 as status bit, referring it needs to be colored. The VGA_Sync circuit checks the status of each pixel in video memory. If the status bit is 1, it is colored in the screen.

G. VGA SYNC Module

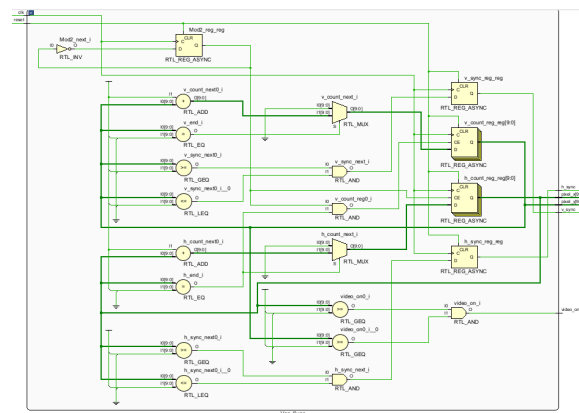


Fig. 14. VGA_SYNC module circuit diagram

The VGA_SYNC Module (Fig. 14) interrupts the VGA monitor with h_sync [Horizontal Sync] signal and v_sync [Vertical Sync] signal. It iterates through every pixel in the screen using pixel_x and pixel_y and checks video buffer memory for each pixels status bit. If the status bit is 1, then the RGB value will be a pre-specified color code which is handled by the master module.

H. MASTER Module

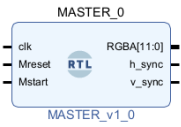


Fig. 15. MASTER module block diagram

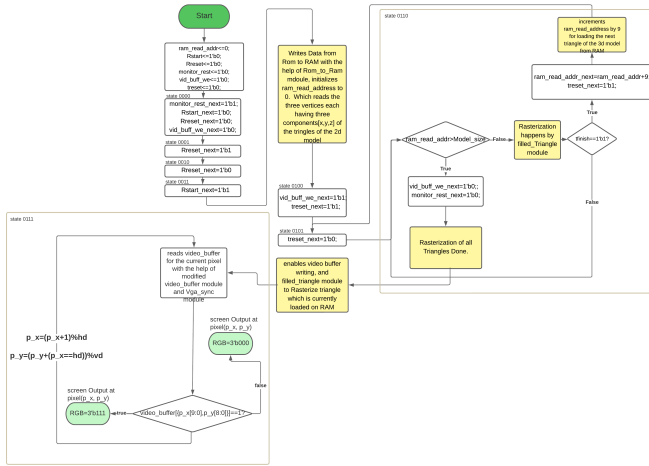


Fig. 16. MASTER module workflow diagram

The MASTER module (Fig. 15) has been designed to utilize all the sub modules discussed before. It organizes each module and outputs final rasterized 2D model in the display screen. The work flow of the overall system which is controlled by master module is shown in Fig. 16

IV. RESULTS

For coding, Visual Studio Code has been used. For simulation, GTKWave [7] is used. Logic synthesis, implementation and bit stream generation has been done in Vivado Design Suite ML Edition software. Targeted device was Nexys 4 board [5] which is equipped with Artix 7 FPGA [6]. The PC where the software were executed, had Windows 10 as the operating system.

A. Filled_Triangle Module Result

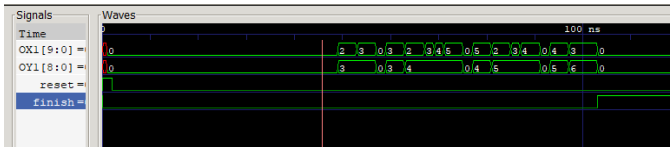


Fig. 17. Simulation output of Filled_Triangle Module

```

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE

PS C:\Users\USER\Desktop\FOR_BOOK\FOR_TB> make
iverilog -o out.vvp filled_tris.v LINE_MODULE.v
#filled_tris.v ROM.v RAM.v ROM_TO_RAM_LOADER.v LINE_MODULE.v Vga_Sync.v master_circ.v video_buffer.v MASTER_TB.v
#Projection.v
vvp out.vvp
filled-  x, x
filled-  0, 0
filled-  2, 3
filled-  3, 3
filled-  0, 0
filled-  3, 3
filled-  2, 4
filled-  3, 4
filled-  4, 4
filled-  5, 4
filled-  0, 0
filled-  5, 4
filled-  2, 5
filled-  3, 5
filled-  4, 5
filled-  0, 0
filled-  4, 5
filled-  3, 6
filled-  0, 0
filled tris.v:321: $finish called at 38368800 (1ps)
#gtkwave test.vcd
PS C:\Users\USER\Desktop\FOR_BOOK\FOR_TB>

```

Fig. 18. Terminal output of Filled_Triangle Module

As we can see from the outputs in Fig. 18 and simulation result of Filled_Triangle module in Fig. 17, it is able to fill a triangle with pixels inside the triangle. It is provided with three vertices of a triangle (2, 2), (5, 4), (3, 6). It provides the filled pixels inside it. It also fills (0, 0) pixel but its a default case.

B. Overall Graphics Processor Result

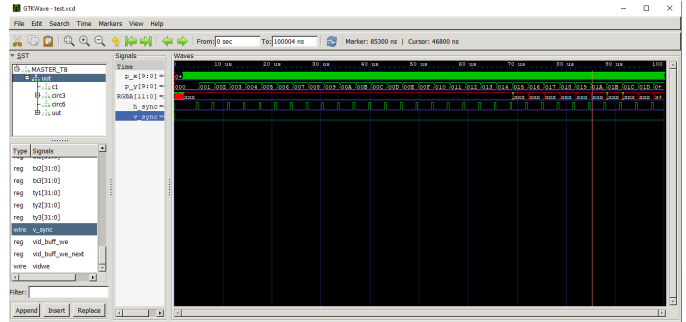


Fig. 19. Simulation Output of The GPU

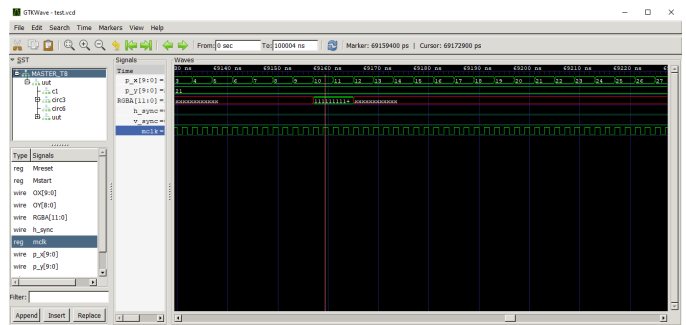


Fig. 20. Simulation Output of The GPU [Zoomed IN]

From the result in Fig. 19 & Fig. 20, it is observable that the GPU can generate RGB values at different pixel position according to the rendered 2D model. It also interfaces the VGA monitor using the h_sync and v_sync signal. In the simulation Fig. 20, the RGB value is 111111111111 at pixel (10,21) and

(11,21). h_sync is 1 at end of the display region in the x axis. v_sync is 1 at end of the frame but it is not observable in the Fig. 20 as the simulation was not run that long.

CONCLUSION

The dynamic integration of FPGA technology into the realm of 2D graphics rendering holds immense promise for both real-time performance enhancements and educational exploration. We have presented a educational GPU platform for implementing graphics related algorithms using FPGA for amateurs and students. In this research work, 2D raster graphics rendering algorithm is implemented correctly on FPGA. Simulation results are verified against numerical values as illustrated in the result section. Our future plan includes, integrating FPU (floating Point Unit) for finer precision and also extending this work to support 3D rendering or other graphics related algorithms in our proposed platform.

REFERENCES

- [1] T. Angelia, "Rendering Techniques in Computer Graphics," Jul. 20, 2020. [Online]. Available: <https://medium.com/@tifangel/rendering-techniques-in-computer-graphics-504e6134fea4>.
- [2] V. Kasik, A. Kurecka and P. Pospech, "3D graphics processing unit with VGA output," IFAC Proceedings Volumes, vol. 46, no. 28, pp. 388-393, 2013.
- [3] J. Schmittler, S. Woop, D. Wagner, W. J. Paul and P. Slusallek, "Realtime ray tracing of dynamic scenes on an FPGA chip," Proceedings of the ACM SIGGRAPH/EUROGRAPHICS conference on Graphics hardware, 2004.
- [4] J. R. Warner, "Real Time 3-D Graphics Processing Hardware Design using Field-Programmable Gate Arrays," Pittsburgh, 2009. [Unpublished].
- [5] Martha, "Nexys 4," Digilent, [Online]. Available: <https://digilent.com/reference/programmable-logic/nexys-4/start>. [Accessed: Sep. 30, 2023].
- [6] "Artix 7 FPGA Family," Xilinx, [Online]. Available: <https://www.xilinx.com/products/silicon-devices/fpga/artix-7.html>. [Accessed: Sep. 30, 2023].
- [7] "Welcome to GTKWave," GTKWave, [Online]. Available: <https://gtkwave.sourceforge.net/>. [Accessed: Sep. 30, 2023].
- [8] D. S. Roy, "Digital System Design," May 21, 2019. [Online]. Available: <https://digitalsystemdesign.in/fpga-implementation-step-by-step/>. [Accessed: Sep. 30, 2023].
- [9] GeeksforGeeks, "Difference between Parallel and Perspective Projection in Computer Graphics," GeeksforGeeks, May 24, 2020. [Online]. Available: <https://www.geeksforgeeks.org/difference-between-parallel-and-perspective-projection-in-computer-graphics/>. [Accessed: Nov. 28, 2023].
- [10] K. Kallio, "Scanline Edge-flag Algorithm for Antialiasing," in TPCG, 2007. [Online]. Available: <https://api.semanticscholar.org/CorpusID:13240124>